KijaniKiosk Capstone: Project Scope

Author: Edwin Otwori | **Track:** A (Infrastructure-First) | **Repository:** `Coder-Edwin/kijanikiosk-devops-foundation`, folder `capstone/` | **Date:** 12 November 2026

1. Problem statement

Every kk-payments release today goes from an engineer's terminal straight into the only environment, the production namespace. Nothing tests a release before customers receive it, and nothing alerts anyone when payments start failing. Verifiable today:

- The Week 8 deployment was a hand-run script that applied to `default`.
- No staging namespace exists in the cluster.
- The repository contains no alert rules.

A broken release or a failed receipt write is currently discovered by a customer, not by the team.

2. In scope (five components)

1. **Staging environment as code.** Terraform creates the `kijani-staging` namespace. Ansible configures it and production with their own ConfigMap and pull secret.
2. **One manifest, two environments.** The same kk-payments Deployment is applied to both namespaces, with readiness and liveness probes, resource limits, structured JSON logs, and environment-specific `DB_HOST` and `RECEIPTS_BUCKET` values from ConfigMaps.
3. **Gated Jenkins pipeline.** A merge to `main` builds and pushes the image, deploys to staging and runs a smoke test. Only then is a production approval offered, and it records the approver and a reason.
4. **Alerting.** A Prometheus alert rule fires when the kk-payments receipt-write error rate exceeds 5% for one minute. The rule is committed to the repository.
5. **Receipt chain integration.** Staging kk-payments writes each receipt to `kk-payments-receipts-staging`. That triggers `kk-receipt-processor` and then `kk-notifier`, running on LocalStack.

3. Out of scope

- **Real AWS.** LocalStack provides the same S3 and Lambda APIs locally, and a free-tier AWS account could not be opened. The application code is unchanged when pointed at real AWS, apart from configuration.
- **A real database.** `DB_HOST` is carried and reported per environment to prove configuration separation. Building a database layer would add risk without testing the pipeline.
- **High availability across nodes.** The cluster is single-node minikube, so node failure is not tested. It is listed as a production gap.
- **Ingress and TLS.** Services stay NodePort for the demo. TLS termination belongs in a later iteration.

4. Success criteria (each shown live in the demo)

#	Criterion	How it is demonstrated
S1	A clean checkout recreates staging in under 5 minutes	Delete `kijani-staging`; run `terraform apply` and the Ansible playbook; show the namespace and ConfigMap
S2	A merge to `main` starts the pipeline with no manual step	Merge a PR; the Jenkins build starts within 2 minutes
S3	Staging serves the new version with staging configuration	Smoke test output shows the new `VERSION-GITSHA` tag, `environment: staging` and the staging `DB_HOST`
S4	Production is gated behind a passing smoke test	A deliberately failing smoke test stops the build before the gate; a passing one shows the gate, and the approver's reason appears in the build log
S5	A staging payment fires the receipt chain	`POST /payments` produces an object in `kk-payments-receipts-staging`, and `kk-notifier` logs it within 10 seconds
S6	Receipt failures raise an alert	Stopping LocalStack makes the receipt error alert fire within 3 minutes; restarting it resolves the alert

5. Architecture

See `docs/architecture.png`. Every arrow is labelled with what flows across it.